

Rapport J16

Jointures relationnelles et diagnostic de couverture

Durée prévue	90 min
Durée réelle	120 min
Écart	+30 min
Base	sql_training
Point de départ Git	4c08e6b

Environnement : Windows 11 / PowerShell / PostgreSQL 17.5 / psql / base sql_training

Point de départ Git : 4c08e6b — docs: complete J15 relational keys normalization and RAW to STAGING migration

1. Objectifs

J16 avait pour objectif d'exploiter le modèle relationnel construit à J15 et de passer d'une simple relation PK/FK à une analyse multi-table orientée qualité des données.

Objectifs atteints :

- exploiter INNER JOIN sur la relation PK/FK ;
- comprendre la cardinalité 1:N ;
- utiliser LEFT JOIN pour préserver la table de référence ;
- détecter les lignes sans correspondance par anti-join ;
- agréger des données après jointure ;
- distinguer COUNT(*) de COUNT(colonne) après LEFT JOIN ;
- construire un KPI de couverture relationnelle ;
- produire une analyse métier avec CASE et COALESCE ;
- tester les jointures avec une donnée temporaire dans une transaction ;
- restaurer l'état permanent avec ROLLBACK ;
- produire et exécuter un script SQL reproductible avec ON_ERROR_STOP=1.

2. Modèle relationnel utilisé

Table parent / référentiel :

analytics.source_systems

- source_id : PRIMARY KEY, IDENTITY ;
- source_code : clé métier unique ;
- source_name ;
- source_category ;
- is_active.

Table enfant normalisée :

staging.training_events

- event_id : PRIMARY KEY ;
- source_id : FOREIGN KEY vers analytics.source_systems(source_id) ;
- event_type ;
- event_value ;
- event_timestamp ;
- ingested_at.

Relation :

analytics.source_systems (1) -> (N) staging.training_events

Plusieurs événements peuvent provenir d'un même système source. La contrainte FOREIGN KEY garantit qu'un source_id stocké dans STAGING possède un parent existant.

3. Validation initiale

La séance a démarré par la vérification de la connexion, de la structure des tables et des volumes.

Résultats :

- staging_events = 15
- distinct_source_ids = 4
- min_event_id = 1
- max_event_id = 18
- source_systems = 4
- active_sources = 4

Une erreur mineure de commande psql a été rencontrée lors d'une première tentative de \d analytics.source_systems, puis corrigée immédiatement.

Capture : J16-01_environment-relational-model-validation.png

4. INNER JOIN sur PK/FK

Requête principale :

```
SELECT
    ste.event_id,
    ste.source_id,
    ss.source_code,
    ss.source_name,
    ste.event_type,
    ste.event_value
FROM staging.training_events AS ste
INNER JOIN analytics.source_systems AS ss
    ON ste.source_id = ss.source_id
ORDER BY ste.event_id ASC;
```

Validation quantitative :

- joined_rows = 15
- distinct_events = 15
- distinct_sources = 4

Conclusion : les 15 événements STAGING trouvent tous un parent dans le référentiel.

Capture : J16-02_inner-join-fk.png

5. Cardinalité et normalisation

Interprétation validée :

- côté 1 : analytics.source_systems ;
- côté N : staging.training_events ;
- plusieurs événements peuvent partager le même source_id ;
- l'intégrité référentielle est imposée par la FOREIGN KEY.

Différence importante avec J15 :

- te.source_system = ss.source_code : correspondance par clé métier textuelle pour résoudre RAW vers le référentiel ;
- ste.source_id = ss.source_id : jointure par clé technique dans le modèle normalisé.

6. LEFT JOIN baseline

Avant d'introduire une source sans événement, le LEFT JOIN retourne :

- left_join_rows = 15
- matched_events = 15
- sources_preserved = 4

Les quatre sources ayant au moins un événement, INNER JOIN et LEFT JOIN donnent alors une couverture apparemment identique.

Capture : J16-03_left-join-baseline.png

7. Source temporaire sans événement

Une transaction contrôlée a été ouverte et une source temporaire a été insérée :

```
BEGIN;

INSERT INTO analytics.source_systems (
    source_code,
    source_name,
    source_category
)
VALUES (
    'weather_api',
    'Weather API',
    'external_api'
);
```

La source existe dans la table parent mais n'a aucun événement enfant.

Résultat :

- weather_api event_count = 0

Capture : J16-04_unmatched-source-test-data.png

8. INNER JOIN versus LEFT JOIN

Avec cinq sources dont weather_api sans événement :

INNER JOIN :

- inner_join_rows = 15
- sources_returned = 4

LEFT JOIN :

- left_join_rows = 16
- matched_events = 15
- sources_preserved = 5

Le LEFT JOIN ajoute une ligne pour weather_api, avec les colonnes de STAGING à NULL.

Capture : J16-05_inner-vs-left-join.png

9. Anti-join : détection des sources sans événement

Pattern utilisé :

```
SELECT
    ss.source_id,
    ss.source_code,
    ss.source_name,
    ste.event_id
FROM analytics.source_systems AS ss
LEFT JOIN staging.training_events AS ste
    ON ss.source_id = ste.source_id
WHERE ste.event_id IS NULL
ORDER BY ss.source_id;
```

Résultat :

- une source sans événement ;
- weather_api détectée correctement.

event_id est un bon marqueur d'absence car cette colonne est PRIMARY KEY et NOT NULL dans la table réelle.

Capture : J16-06_unmatched-source-detection.png

10. Agrégations après LEFT JOIN

Résultats :

```
| Source | event_count | average_value | total_value |
| microgrid_poc | 7 | 190.99 | 1336.90 |
| sensor_gateway | 4 | 28.00 | 112.00 |
```

| tomra_training | 3 | 3.00 | 9.00 |

| manual_psql | 1 | 125.50 | 125.50 |

| weather_api | 0 | NULL | NULL |

Pour weather_api, COUNT(ste.event_id) retourne 0, alors que AVG() et SUM() retournent NULL puisqu'aucune valeur non NULL n'est disponible.

Comparaison supplémentaire :

- COUNT(*) = 1 pour weather_api ;

- COUNT(ste.event_id) = 0.

Cela démontre pourquoi COUNT(*) peut créer un KPI incorrect après un LEFT JOIN.

Capture : J16-07_join-aggregation-and-count-comparison.png

11. KPI de couverture relationnelle

Une CTE source_activity a d'abord produit une ligne par source, puis un second niveau d'agrégation a calculé le KPI.

Résultats :

- total_sources = 5

- sources_with_events = 4

- sources_without_events = 1

- coverage_rate_pct = 80.00

La division a été protégée avec :

```
NULLIF(COUNT(*), 0)
```

afin d'éviter une division par zéro si le référentiel était vide.

Capture : J16-08_relational-coverage-kpi.png

12. Challenge autonome

Besoin métier : produire pour chaque système :

- source_name

- source_category

- event_count

- average_value

- total_value

- activity_status

Règles :

- event_count > 0 -> ACTIVE

- event_count = 0 -> NO EVENTS

- valeurs agrégées absentes -> 0.00

La première tentative retournait seulement quatre lignes et perdait Weather API. Elle arrondissait aussi certaines valeurs à l'entier. Le diagnostic a porté sur la construction du CTE, le type de jointure et la gestion des agrégats NULL.

Version validée :

```
WITH source_activity AS (
  SELECT
    ss.source_name,
    ss.source_category,
    COUNT(ste.event_id) AS event_count,
    COALESCE(ROUND(AVG(ste.event_value), 2)::numeric, 0.00) AS average_value,
    COALESCE(ROUND(SUM(ste.event_value), 2)::numeric, 0.00) AS total_value
  FROM analytics.source_systems AS ss
  LEFT JOIN staging.training_events AS ste
    ON ss.source_id = ste.source_id
  GROUP BY
    ss.source_id,
    ss.source_name,
    ss.source_category
)
SELECT
  source_name,
```

```

    source_category,
    event_count,
    average_value,
    total_value,
    CASE
        WHEN event_count > 0 THEN 'ACTIVE'
        WHEN event_count = 0 THEN 'NO EVENTS'
        ELSE 'UNKNOWN'
    END AS activity_status
FROM source_activity
ORDER BY event_count DESC, source_name ASC;

```

Résultat final :

- Microgrid POC — 7 — 190.99 — 1336.90 — ACTIVE
- Sensor Gateway — 4 — 28.00 — 112.00 — ACTIVE
- TOMRA Training — 3 — 3.00 — 9.00 — ACTIVE
- Manual PSQL — 1 — 125.50 — 125.50 — ACTIVE
- Weather API — 0 — 0.00 — 0.00 — NO EVENTS

Capture : J16-10_autonomous-relational-analysis-corrected.png

13. Incidents, erreurs et résolutions

13.1 Transaction laissée ouverte et perte de connexion

Une transaction de test est restée ouverte pendant une interruption prolongée. La connexion PostgreSQL a ensuite été perdue :

server closed the connection unexpectedly

psql s'est reconnecté automatiquement. La transaction non commitée a été annulée par PostgreSQL.

Validation après reconnexion :

- quatre sources permanentes ;
- weather_api absente.

Capture : J16-09_transaction-loss-rollback-validation.png

Leçon : une transaction ouverte ne doit pas être laissée inutilement inactive. En cas de fin de session, les modifications non commitées sont annulées.

13.2 Transaction en état aborted

Lors d'une reprise, une erreur antérieure dans la transaction a placé celle-ci en état d'échec. Le prompt est devenu :

sql_training=!#

Les commandes suivantes ont produit :

ERROR: current transaction is aborted, commands ignored until end of transaction block

Résolution :

1. ROLLBACK;
2. vérification du retour à quatre sources ;
3. nouvelle transaction ;
4. réinsertion temporaire de weather_api ;
5. réexécution du challenge.

Repères psql retenus :

- sql_training=# : aucune transaction explicite active ;
- sql_training=*# : transaction active et valide ;
- sql_training=!# : transaction active mais échouée, ROLLBACK requis.

13.3 ROLLBACK et séquence IDENTITY

Pendant l'exécution finale du script, weather_api a reçu un source_id supérieur aux précédents essais. Cela montre qu'un ROLLBACK annule les données de la transaction mais ne remet pas nécessairement la séquence/identity à sa valeur précédente.

14. Restauration finale du dataset

Après le challenge :

ROLLBACK;

Validation :

- source_systems = 4
- active_sources = 4
- staging_events = 15
- distinct_source_ids = 4
- min_event_id = 1
- max_event_id = 18
- weather_api = 0 row

Capture : J16-11_final-rollback-dataset-restoration.png

15. Script reproductible

Fichier :

documentation/semaine-03/jour-16/code/J16_relational_joins_coverage.sql

Le script automatise :

1. validation initiale ;
2. INNER JOIN ;
3. LEFT JOIN baseline ;
4. transaction contrôlée ;
5. insertion temporaire de weather_api ;
6. comparaison INNER/LEFT ;
7. anti-join ;
8. agrégations ;
9. COUNT(*) vs COUNT(event_id) ;
10. KPI de couverture ;
11. analyse métier ;
12. ROLLBACK ;
13. validation finale.

Commande d'exécution :

```
psql -U postgres -d sql_training `
-v ON_ERROR_STOP=1 `
-f ".\documentation\semaine-03\jour-16\code\J16_relational_joins_coverage.sql"
```

ON_ERROR_STOP=1 force psql à arrêter le script à la première erreur.

Validation finale du script :

- exécution complète sans erreur ;
- J16 COMPLETE affiché ;
- quatre sources permanentes ;
- 15 événements ;
- weather_api_rows = 0.

Captures :

- J16-12_script-validation-0.png
- J16-12_script-validation-1.png
- J16-12_script-validation-2.png
- J16-12_script-validation-3.png

16. Inventaire des preuves

Captures présentes :

1. J16-01_environment-relational-model-validation.png
2. J16-02_inner-join-fk.png
3. J16-03_left-join-baseline.png
4. J16-04_unmatched-source-test-data.png
5. J16-05_inner-vs-left-join.png
6. J16-06_unmatched-source-detection.png
7. J16-07_join-aggregation-and-count-comparison.png
8. J16-08_relational-coverage-kpi.png
9. J16-09_transaction-loss-rollback-validation.png
10. J16-10_autonomous-relational-analysis-corrected.png
11. J16-11_final-rollback-dataset-restoration.png
12. J16-12_script-validation-0.png
13. J16-12_script-validation-1.png
14. J16-12_script-validation-2.png
15. J16-12_script-validation-3.png

Code :

• J16_relational_joins_coverage.sql

17. KPI de progression J16

| Compétence | État |

| Relation PK/FK | Validé |

| Cardinalité 1:N | Validé |

| INNER JOIN | Validé |

| LEFT JOIN | Validé |

| Anti-join | Validé |

| Agrégation multi-table | Validé |

| COUNT(*) vs COUNT(colonne) | Validé |

| KPI de couverture | Validé |

| CTE | Validé |

| CASE / COALESCE | Validé |

| Transaction contrôlée | Validé |

| Diagnostic transaction aborted | Validé |

| ROLLBACK / restauration | Validé |

| Script SQL reproductible | Validé |

| Challenge autonome | Validé après diagnostic et correction |

18. Git et clôture

Avant commit, le dossier J16 doit être ajouté et vérifié :

```
git add .\documentation\semaine-03\jour-16\  
git status  
git diff --cached --stat  
git diff --cached --name-status
```

Commit prévu :

```
git commit -m "docs: complete J16 SQL relational joins and coverage analysis"  
git push
```

Validation finale :

```
git status  
git log --oneline --decorate -8
```

La journée ne sera considérée définitivement archivée qu'après validation du commit et synchronisation main / origin/main.

19. Bilan temporel

- Durée prévue : **90 min**
- Durée réelle : **120 min**
- Écart : **+30 min**
- Timer arrêté avant génération finale des rapports et opérations Git ; ces opérations postérieures ne sont pas ajoutées à la durée réelle.

20. Conclusion

J16 marque le passage d'un modèle relationnel construit à J15 à son exploitation analytique. Les jointures ne sont plus utilisées uniquement pour rapprocher des tables : elles deviennent des outils de contrôle de couverture, de détection d'anomalies et de construction de KPI.

La séance a également apporté une expérience pratique importante des transactions PostgreSQL : rollback automatique lors d'une perte de session, transaction aborted, lecture des prompts psql et restauration contrôlée du dataset.

21. Prochaines étapes

Après clôture documentaire et Git de J16 :

1. exécuter J17 intégralement avec son propre timer ;
2. conserver la séparation documentaire entre J16 et J17 ;
3. poursuivre ensuite le jour correspondant du plan sans dépasser J112.

22. Potentiel de réutilisation

J16 peut être transformé en :

- tutoriel : **INNER JOIN vs LEFT JOIN avec PostgreSQL** ;
- vidéo : **Pourquoi COUNT(*) peut fausser un KPI après LEFT JOIN** ;
- exercice : **détecter les référentiels sans données avec un anti-join** ;
- module de formation : **PK/FK, cardinalité et jointures relationnelles** ;
- atelier Data Quality : **mesurer un taux de couverture relationnelle** ;
- démonstration PostgreSQL : **BEGIN, transaction aborted, ROLLBACK et comportement des séquences IDENTITY**.

Le script `J16_relational_joins_coverage.sql` constitue l'actif technique reproductible associé à ce module.